Oxford SM4000 Cryostat KULT

Public API and Clarius Cycle Guide

Purpose

Build reproducible Clarius subsites from the public Oxford KULT modules, with the Temperature-and-Field cycle as the canonical project pattern.

Audience

4200A-SCS users who consume the compiled `Oxford_SM4000_Cryo` user library. No knowledge of the Win32 transport, ISOBUS framing, or private header implementation is required.

Main recipe	Temperature-and-Field subsite cycle
Runtime	Clarius on the Keithley 4200A-SCS
Library	`Oxford_SM4000_Cryo`
Document date	13 November 2026

Contents

1 Public API model **2**
1.1 Module summary . . . 2

2 Canonical Temperature-and-Field project **3**
2.1 Create the project . . . 3
2.2 Attach an action to the library . . . 3
2.3 Enter paired schedules . . . 3
2.4 Execution order . . . 3

3 Public module reference **5**
3.1 Initialization and operating mode . . . 5
3.2 Scheduled environment setters . . . 5
3.3 Combined readout . . . 5
3.4 Individual read modules . . . 6

4 Demo patterns **6**
4.1 One-point operation without cycling . . . 6
4.2 Read-only monitoring cycle . . . 6
4.3 Focused device demo . . . 6

5 Building a custom cycle safely **6**

6 Return status and common configuration errors **7**

1 Public API model

The Oxford library is consumed through Clarius *Custom Actions*. Each action selects a public module from the `Oxford_SM4000_Cryo` user library. The project tree determines execution order; a cycling Subsite supplies the current cycle number.

The cycle contract

The scheduled setters read the current Clarius cycle through `GetKiteCycle()`. Cycle 1 selects array row 1, cycle 2 selects row 2, and so on. Outside a cycling Subsite, Clarius reports cycle 1, so the same modules also work normally with a one-element array.

The public API exposes measurement values and the selected schedule values. It does not expose transport stages, Windows error numbers, build markers, or private device structures. Every module has return type `int`; **0 = success**. A negative return value indicates that the action did not complete.

1.1 Module summary

Module	Role	Use in a project
`oxford_initialize`	Checks the four Oxford devices and applies the IPS field sweep-rate initialization from the reference controller.	Place once before the cycling Subsite. Do not place it inside the cycle.
`oxford_set_temperature`	Selects a target from the temperature schedule, configures the ITC503, and waits for the reference stability criterion.	Place near the start of each environment cycle.
`oxford_set_magnetic_field`	Selects a target from the field schedule and performs the reference persistent-magnet procedure.	Place after temperature and before electrical measurements.
`oxford_read_all`	Returns the combined ILM, ITC, Lambda, and IPS sensor snapshot used by the reference controller.	Preferred read action when all four Oxford devices are connected.
`oxford_read_itc`	Complete ITC503 snapshot.	Use for focused temperature monitoring or debugging.
`oxford_read_ilm`	Complete ILM200 snapshot.	Use for level monitoring.
`oxford_read_lambda`	Lambda controller snapshot.	Use when Lambda values are required independently.
`oxford_read_ips`	Complete IPS120 snapshot.	Use for detailed magnet and power-supply monitoring.
`oxford_set_active`	Sets the ILM probe to the reference active mode.	Optional project setup action.
`oxford_set_standby`	Applies the reference standby sequence and requests zero magnetic field.	Optional final project action.

2 Canonical Temperature-and-Field project

The supported main recipe is one saved Clarius project containing one Temperature-and-Field Subsite. The environment setters, Oxford readback, and Keithley electrical test are ordered within that Subsite.

```
Oxford Temperature and Field Cycle
|-- Initialize
`-- Site
    `-- Temperature and Field Cycle        [Subsite: Cycle]
        |-- Set Temperature
        |-- Set Magnetic Field
        |-- Read All
        `-- Device / Keithley measurement
```

The Site shown here matches the example project and is useful when the project also contains devices and electrical tests. The Oxford actions themselves do not require wafer probing.

2.1 Create the project

1. Create and load a new blank Clarius project.
2. From **Select** → **Actions**, drag a **Custom Action** to the project root. Rename it **Initialize** and attach `oxford_initialize`.
3. From **Select** → **Wafer Plan**, add a Site and a Subsite. Rename the Subsite **Temperature and Field Cycle**.
4. Add three Custom Actions beneath the Subsite, in this order: **Set Temperature**, **Set Magnetic Field**, and **Read All**.
5. Attach the corresponding public modules from `Oxford_SM4000_Cryo`.
6. Add the Keithley device and electrical test after **Read All**.
7. Select the Subsite, choose **Configure**, set **Subsite Operation** to **Cycle**, and set **Number of Cycles** to the schedule length.

2.2 Attach an action to the library

For every Custom Action, select the action, open **Configure**, and use the right-hand **Test Settings** pane. Select `Oxford_SM4000_Cryo` under **User Libraries**, then select the module under **User Modules**. Figure 1 shows the exact location of both selectors.

2.3 Enter paired schedules

The temperature and field schedules are paired by row. They are not two independent nested loops. With four Subsite cycles, for example:

Cycle	temperature_k	field_t
1	4.2 K	0.0 T
2	4.2 K	1.0 T
3	10.0 K	0.0 T
4	10.0 K	1.0 T

Enter the temperature list in **Set Temperature** and the field list in **Set Magnetic Field**. The number of populated rows in both lists must be at least the configured number of Subsite cycles.

Do not leave blank rows between schedule values. Repeat the same procedure for `field_t`. For a one-point experiment, enter one value in each array and set the Subsite to one cycle.

2.4 Execution order

For each Subsite cycle, Clarius executes the checked children from top to bottom:

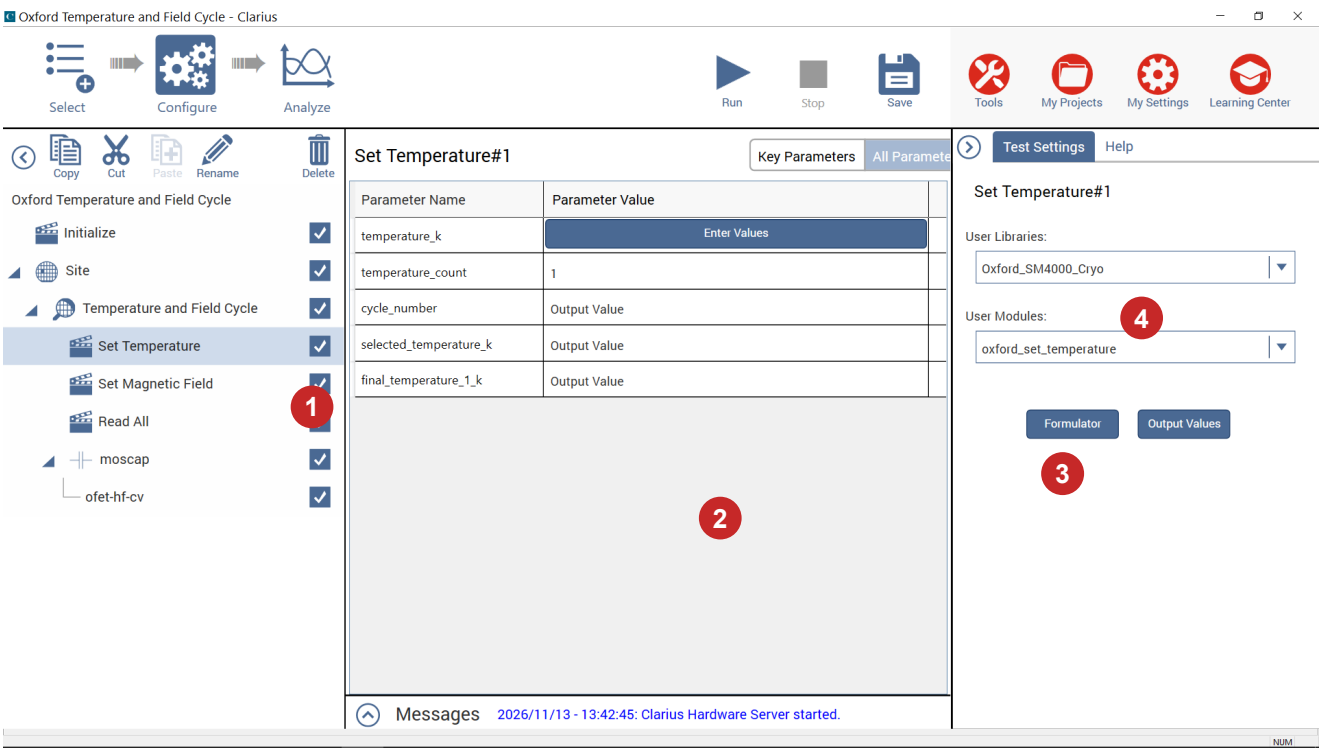


Figure 1: Configuring the Set Temperature action. 1: action location in the project tree; 2: array input in Key Parameters; 3: user library and public module selection; 4: Output Values selection.

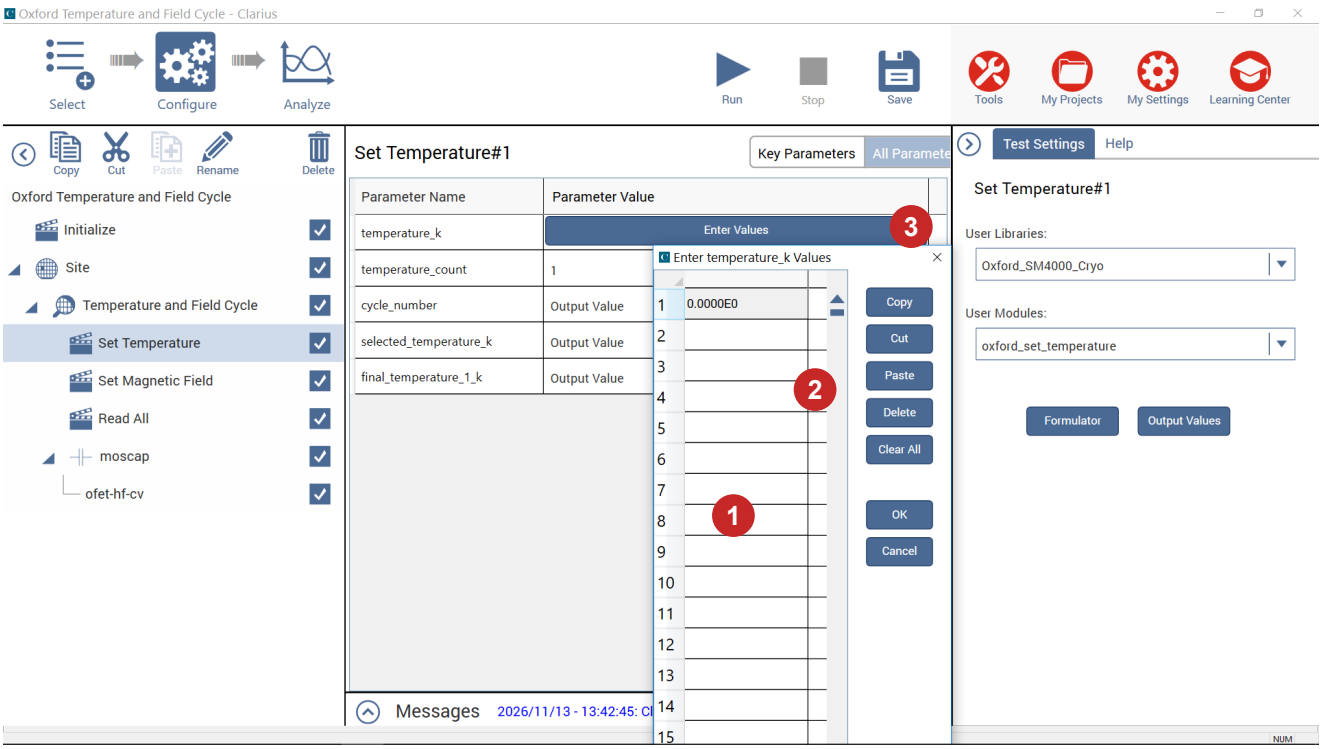


Figure 2: Editing the temperature schedule. 1: Enter Values control; 2: one target per contiguous row; 3: confirm the list with OK. The `temperature_count` value follows the populated input array.

1. `oxford_set_temperature` selects the current temperature row, configures the ITC503, and blocks until the rolling temperature-error condition from the reference controller is satisfied.
2. `oxford_set_magnetic_field` selects the corresponding field row and completes the persistent-field procedure.
3. `oxford_read_all` records the Oxford environment immediately before the electrical measurement.
4. The Keithley device test runs at the established environment point.

Operational note

The magnetic-field action contains the reference 60-second switch-heater waits and returns the source toward zero after establishing persistent field. It can therefore take substantially longer than the temperature action or a normal read action. Do not add it to a monitoring-only cycle.

3 Public module reference

3.1 Initialization and operating mode

Module	Inputs	Outputs and behaviour
`oxford_initialize`	None	`cycle_number`. Checks ILM200, ITC503, Lambda, and IPS120 communication; sets the IPS field sweep rate to 0.3382 T/min.
`oxford_set_active`	None	`cycle_number`. Sets the ILM probe to FAST using the reference active sequence.
`oxford_set_standby`	None	`cycle_number`, `final_source_field_t`. Sets ITC gas and heater outputs to manual zero, changes the ILM probe to SLOW, and requests zero magnetic field.

3.2 Scheduled environment setters

Module	Inputs	Outputs
`oxford_set_temperature`	`temperature_k` (double array), `temperature_count` (array size)	`cycle_number`, `selected_temperature_k`, `final_temperature_1_k`
`oxford_set_magnetic_field`	`field_t` (double array), `field_count` (array size)	`cycle_number`, `selected_field_t`, `final_source_field_t`

The count parameters are created by KULT as the associated array-size parameters. The schedule index is `cycle_number-1`. If the current cycle exceeds the corresponding count, the module returns `-21012`.

3.3 Combined readout

`oxford_read_all` has no inputs. It returns `cycle_number` and the same sensor groups collected by the reference controller:

Device	Output names
ILM200	`helium_percent`, `nitrogen_percent`
ITC503	`temperature_1_k`, `temperature_2_k`, `temperature_3_k`, `temperature_setpoint_k`, `temperature_error_k`, `gas_percent`, `heater_volts`

Device	Output names
Lambda	`lambda_pressure_mbar`, `lambda_temperature_k`, `lambda_valve_percent`
IPS120	`current_output_a`, `current_setpoint_a`, `magnet_current_a`, `persistent_current_a`, `trip_current_a`, `field_output_t`, `field_setpoint_t`, `persistent_field_t`, `trip_field_t`, `supply_voltage_v`, `switch_heater_current_ma`

Use the action's **Output Values** button to select only the columns that should be promoted into the Subsite Analyze sheet.

3.4 Individual read modules

The individual readers have no inputs and always return `cycle_number`. They are useful for focused projects and for installations where not all four Oxford devices are connected.

Module	Output groups
`oxford_read_ilm`	Channel usage/status, relay status, helium and nitrogen levels, channel currents, and valve position.
`oxford_read_itc`	ITC state, temperatures 1–3, setpoint, error, heater percentage/voltage, gas percentage, and P/I/D settings.
`oxford_read_lambda`	Lambda state, pressure, temperature, and valve position.
`oxford_read_ips`	IPS state; current, field, sweep, persistent, trip, voltage, switch-heater, lead-resistance, and magnet-inductance values.

4 Demo patterns

These are demonstrations of the same public API, not additional required projects.

4.1 One-point operation without cycling

Place `oxford_set_temperature`, `oxford_set_magnetic_field`, and `oxford_read_all` directly in a project. Enter one temperature and one field value. With no active Subsite cycle, `GetKiteCycle()` returns 1, so the first array row is used.

4.2 Read-only monitoring cycle

Create a cycling Subsite containing only `oxford_read_all` and any desired Keithley measurement. Set the number of cycles and cycle delay in the Subsite configuration. This records the environment without changing temperature or field.

4.3 Focused device demo

For initial commissioning, use `oxford_read_itc` and `oxford_read_ilm` independently. This avoids making a combined read depend on Lambda and IPS availability.

5 Building a custom cycle safely

Users may rearrange the public modules, but should preserve the following rules:

1. Put `oxford_initialize` outside the cycling Subsite.
2. Keep environment-changing actions before electrical measurements.
3. Keep temperature and field schedules contiguous and at least as long as the Subsite cycle count.

4. Treat schedule rows as paired environment points.
5. Use `oxford_read_all` immediately before the electrical test when a synchronized environment record is required.
6. Use individual readers when only part of the Oxford system is connected.
7. Preserve project recipes by copying the Clarius project before changing schedules. The compiled KULT library is shared; action parameter values belong to the project instance.

6 Return status and common configuration errors

The module return value is the public status interface. There are no separate build, transport-stage, or Windows-error output parameters.

Status	Meaning and response
0	Action completed successfully.
-21009	An Oxford device returned a response beginning with “?”. Check the command applicability and device operating state.
-21012	The active cycle number exceeds the schedule length. Increase the array length or reduce the Subsite cycle count.
Other negative value	Communication, response, configuration, or close failure. Confirm COM1, cable power through DTR/RTS, and the connected device.

Recommended commissioning sequence

First run `oxford_initialize`; then run `oxford_read_itc` and `oxford_read_ilm`; then test one temperature-field point; finally enable the complete schedule and electrical measurement.